

# Einführung in R

## Data Analysis Bootcamp

Dr. Alexander Bauer  
Essential Data Science

01.10.2026

# Gliederung

- 1 Grundlagen
- 2 Objektklassen
- 3 Funktionen und Pakete
- 4 Arbeit mit Daten
- 5 Weitere Materialien

# Grundlagen

- 1 Grundlagen
- 2 Objektklassen
- 3 Funktionen und Pakete
- 4 Arbeit mit Daten
- 5 Weitere Materialien

# Grundlagen – Was ist R?

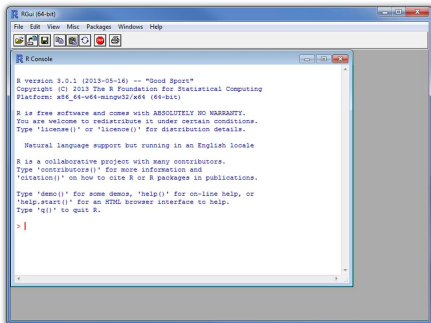
R ist ...

- eine Statistik-Software
- eine Programmiersprache
- verfügbar für alle Betriebssystemen
- erweiterbar durch das Laden verschiedener packages
- state-of-the-art in wissenschaftlicher Forschung
- kostenlos und open source
- die Software der Wahl einer großen Community von Nutzern aus allen (wissenschaftlichen) Bereichen

# Grundlagen – Installation von R

## Installation:

- 1 R von <https://cran.r-project.org/> herunterladen



- 2 Tipp: Mit der Basis-Software R zu arbeiten ist wenig angenehm. Es empfiehlt sich einen Editor zu benutzen, z.B. RStudio: <https://posit.co/download/rstudio-desktop/>

# Grundlagen – RStudio

The screenshot shows the RStudio environment with the following components highlighted:

- 1- Code Editor:** The top-left pane containing R code for loading ggplot2, viewing the 'diamonds' dataset, and creating a scatter plot of carat vs price.
- 2- R Console:** The bottom-left pane showing the output of the code, including summary statistics for the 'diamonds' dataset and the execution of the plot command.
- 3- Workspace and History:** The top-right pane showing the 'diamonds' dataset loaded into the workspace and the history of executed commands.
- 4- Plots and files:** The bottom-right pane displaying a scatter plot titled 'Diamond Pricing' showing the relationship between 'Carat' (x-axis) and 'Price' (y-axis), colored by clarity.

Quelle: <http://www.sthda.com/english/wiki/running-rstudio-and-setting-up-your-working-directory-easy-r-programming>

# Objektklassen

- 1 Grundlagen
- 2 Objektklassen**
- 3 Funktionen und Pakete
- 4 Arbeit mit Daten
- 5 Weitere Materialien

# Objektklassen

Jedes Objekt in R hat eine spezifische Klasse. Beispiele:

```
> class(17)
[1] "numeric"
> class("0'zapft is!")
[1] "character"
```

In einem **Vektor** besitzen alle Elemente die selbe Klasse:

```
> a <- c("Ich bin", "ein Vektor mit", 4, "Elementen")
> class(a[3])
[1] "character"
```

In einer **Liste** besitzt jedes Element eine individuelle Klasse:

```
> b <- list("Ich bin", "eine Liste mit", 4, "Elementen")
> class(b[[3]])
[1] "numeric"
```

# Objektklassen – Type conversion

Mathematische Operationen können nur auf Objekte mit bestimmten Klassen angewendet werden:

```
> 2 + 2
[1] 4
> 2 + "2"
Error in 2 + "2": non-numeric argument
```

Per **type conversion** kann man die Klasse eines Objektes ändern:

```
> 2 + as.numeric("2")
[1] 4
# Auch möglich: as.character, as.factor, as.Date, ...
```

# Objektklassen

Betrachten wir nun Tabellen:

In einer **Matrix** besitzen alle Elemente die selbe Klasse:

```
> m <- matrix(c("Wort",1,"Wort",9), nrow = 2, byrow = TRUE)
> m
      [,1] [,2]
[1,] "Wort"  "1"
[2,] "Wort"  "9"
> class(m[,2]) # zweite Spalte
[1] "character"
```

In einem **data frame** besitzt jede Spalte eine individuelle Klasse:

```
> d <- data.frame(Var1 = c("Wort","Wort"), Var2 = c(2,4))
> d
  Var1 Var2
1 Wort    2
2 Wort    4
> class(d[,2]) # zweite Spalte
[1] "numeric"
```

# Funktionen und Pakete

- 1 Grundlagen
- 2 Objektklassen
- 3 Funktionen und Pakete**
- 4 Arbeit mit Daten
- 5 Weitere Materialien

# Benutzung von Funktionen

## Arbeit mit Funktionen

- Für bestehende Funktionen: Hilfeseiten benutzen

```
> ?mean
```

- Aufruf einer Funktion:

```
> sample <- c(2,5,3,17)
> mean(sample)
[1] 6.75
```

- Definition einer neuen Funktion:

```
> do_something <- function(a, b) {
+   result <- a + b
+   return(result)
+ }
> do_something(2,4)
[1] 6
```

# Benutzung von Paketen

## Arbeit mit Paketen

- 1 Finden eines Pakets mit der gewünschten Funktion, z.B. per Google
- 2 Installation des Pakets: *nur ein einziges Mal notwendig*

```
> install.packages("paketXY")
```

- 3 Laden des Pakets: *bei jedem Neustart von R notwendig*

```
> library("paketXY")
```

# Arbeit mit Daten

- 1 Grundlagen
- 2 Objektklassen
- 3 Funktionen und Pakete
- 4 Arbeit mit Daten**
- 5 Weitere Materialien

# Arbeit mit Daten

## Daten einlesen:

```
> dat <- read.table(  
+   file      = "Daten/WDI_Daten.csv",  
+   header    = TRUE,  
+   sep       = ",",  
+   dec       = ".")
```

## Mit Variablen arbeiten:

```
> mean(dat$Area)  
[1] 491912.6  
> dat$new_variable <- dat$CO2emission / dat$Area
```

## Daten kennenlernen mit str():

```
> str(dat)
'data.frame': 315 obs. of 8 variables:
 $ country      : Factor w/ 45 levels "Albania","Andorra",...: 2 2 2 2
   2 2 2 1 1 1 ...
 $ region       : Factor w/ 4 levels "Eastern Europe",...: 3 3 3 3 3 3
   3 3 3 3 ...
 $ year         : int  2005 2006 2007 2008 2009 2010 2011 2005 2006
   2007 ...
 $ CO2emission: num  576 546 539 539 517 ...
 $ Population  : int  81223 83373 84878 85616 85474 84419 82326
   3011487 2992547 2970017 ...
 $ Area        : int  470 470 470 470 470 470 470 27400 27400 27400
   ...
 $ BevDichte   : num  173 177 181 182 182 ...
 $ LandJahr    : Factor w/ 315 levels "Albania_2005",...: 8 9 10 11
   12 13 14 1 2 3 ...
```

# Arbeit mit Daten - Visualisierung

Hier einzelne Beispiele auf Basis des ggplot2 Pakets:

```
> library(ggplot2)
> theme_set(theme_bw()) # globales Plot-Theme setzen

> ggplot(dat, aes(x = region)) +
+   geom_bar()

> ggplot(dat, aes(x = year, y = CO2emission)) +
+   geom_point()

> ggplot(dat, aes(x = year, y = CO2emission, color = country)) +
+   geom_line()
```

Auf der Suche nach Inspiration? Siehe die [R Graph Gallery](#).

# Arbeit mit Daten - Best Practices

## Workflow für eine Analyse:

- 1 Erstelle eines Projektordners in welchen man Code und Daten legt
- 2 Arbeite mit einer .R-Datei und nicht in der Konsole
- 3 Erster Schritt: Setze das Arbeitsverzeichnis per `setwd()`

Anm.: In Windows muss man die Slashes im Pfad umdrehen!

- 4 Erster Schritt nach dem Einlesen der Daten:  
Stelle sicher, dass jede Variable die korrekte Klasse hat.  
Falls nicht der Fall, nutze `as.numeric()`, `as.factor()` etc.
- 5 Führe die Datenaufbereitung komplett in R durch (automatische Reproduzierbarkeit!) und nicht in Excel
- 6 Nutze Kommentare im Code, um zu beschreiben was der Code macht!

# Weitere Materialien

- 1 Grundlagen
- 2 Objektklassen
- 3 Funktionen und Pakete
- 4 Arbeit mit Daten
- 5 Weitere Materialien**

# Weitere Materialien

Zusätzliche hilfreiche Materialien:

- [Getting started in R](#)
- [Cheatsheets](#) von Posit

## Weitere Fragen?

⇒ fragt die KI eures Vertrauens